

Write Testable Code with Moq

Objective

The objective of this exercise is to understand how to write testable code using Moq and NUnit. The application contains a mail sending module that communicates with an SMTP server. To avoid sending actual emails during testing, Moq is used to create mock objects for external dependencies. This enables faster test execution and promotes loose coupling between components.

Task 1: Creating the Class Library

Step 1: Create CustomerCommLib Project

1. Open Visual Studio 2022.
2. Click **Create a New Project**.
3. Select **Class Library (.NET)**.
4. Enter the project name as **CustomerCommLib**.
5. Select **.NET 8.0** framework and create the project.

Step 2: Create IMailSender Interface

Create a new interface named **IMailSender.cs**.

```
namespace CustomerCommLib
{
    public interface IMailSender
    {
        bool SendMail(string toAddress, string message);
    }
}
```

Step 3: Create MailSender Class

Create a new class named **MailSender.cs**.

```
namespace CustomerCommLib
{
    public class MailSender : IMailSender
    {
        public bool SendMail(string toAddress, string message)
        {
            return true;
        }
    }
}
```

```
    }  
}  
}
```

Step 4: Create CustomerComm Class

Create a new class named **CustomerComm.cs**.

```
namespace CustomerCommLib
```

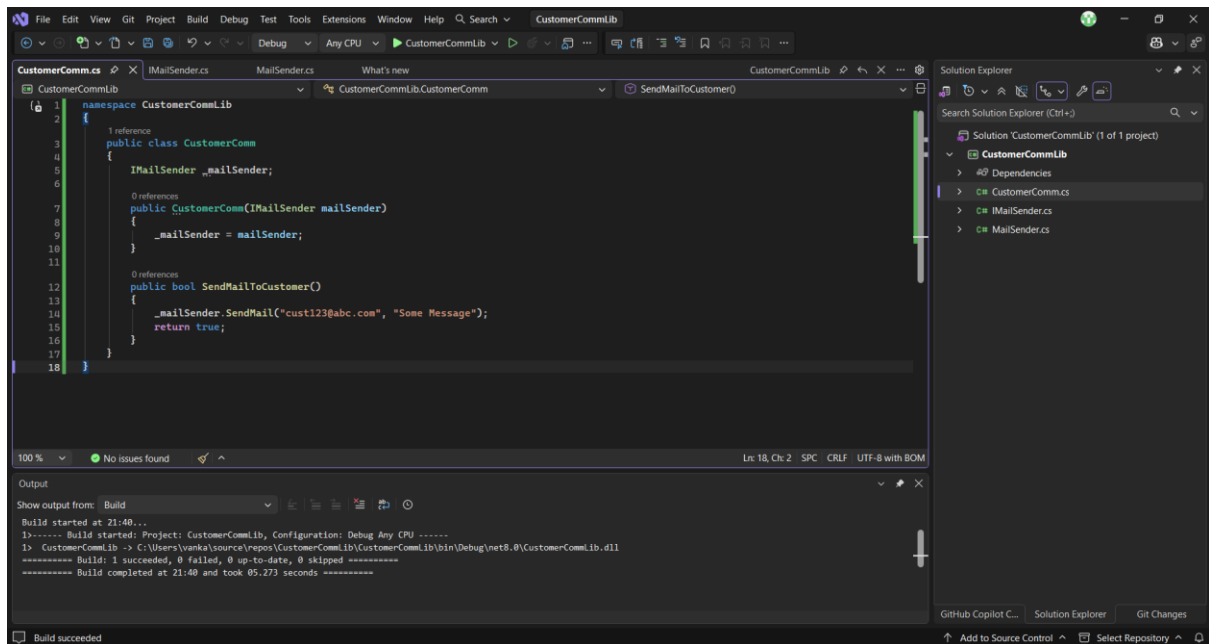
```
{  
    public class CustomerComm  
    {  
        private IMailSender _mailSender;  
  
        public CustomerComm(IMailSender mailSender)  
        {  
            _mailSender = mailSender;  
        }  
  
        public bool SendMailToCustomer()  
        {  
            _mailSender.SendMail("cust123@abc.com", "Some Message");  
            return true;  
        }  
    }  
}
```

Step 5: Build the Project

1. Save all files.
2. Click **Build → Rebuild Solution**.
3. Verify that the build is successful.

Output

Build completed successfully without errors.



Task 2: Creating Unit Test Project using NUnit and Moq

Step 1: Create Test Project

1. Right-click the solution.
2. Select **Add → New Project**.
3. Choose **NUnit Test Project**.
4. Name the project **CustomerComm.Tests**.

Step 2: Add Project Reference

1. Right-click **CustomerComm.Tests**.
2. Select **Add → Project Reference**.
3. Check **CustomerCommLib**.
4. Click **OK**.

Step 3: Install Required Packages

Using NuGet Package Manager, install:

- NUnit
- NUnit3TestAdapter
- Moq

Step 4: Create Unit Test Class

Delete the default **UnitTest1.cs** file and create a new class named **CustomerCommTest.cs**.

using NUnit.Framework;

```

using Moq;

using CustomerCommLib;

namespace CustomerComm.Tests
{
    [TestFixture]
    public class CustomerCommTest
    {
        private Mock<IMailSender> mockMailSender;

        [OneTimeSetUp]
        public void Init()
        {
            mockMailSender = new Mock<IMailSender>();

            mockMailSender
                .Setup(x => x.SendMail(It.IsAny<string>(), It.IsAny<string>()))
                .Returns(true);
        }

        [TestCase]
        public void SendMailToCustomer_ShouldReturnTrue()
        {
            CustomerComm customerComm =
                new CustomerComm(mockMailSender.Object);

            bool result = customerComm.SendMailToCustomer();

            Assert.That(result, Is.True);
        }
    }
}

```

}

Step 5: Execute the Test

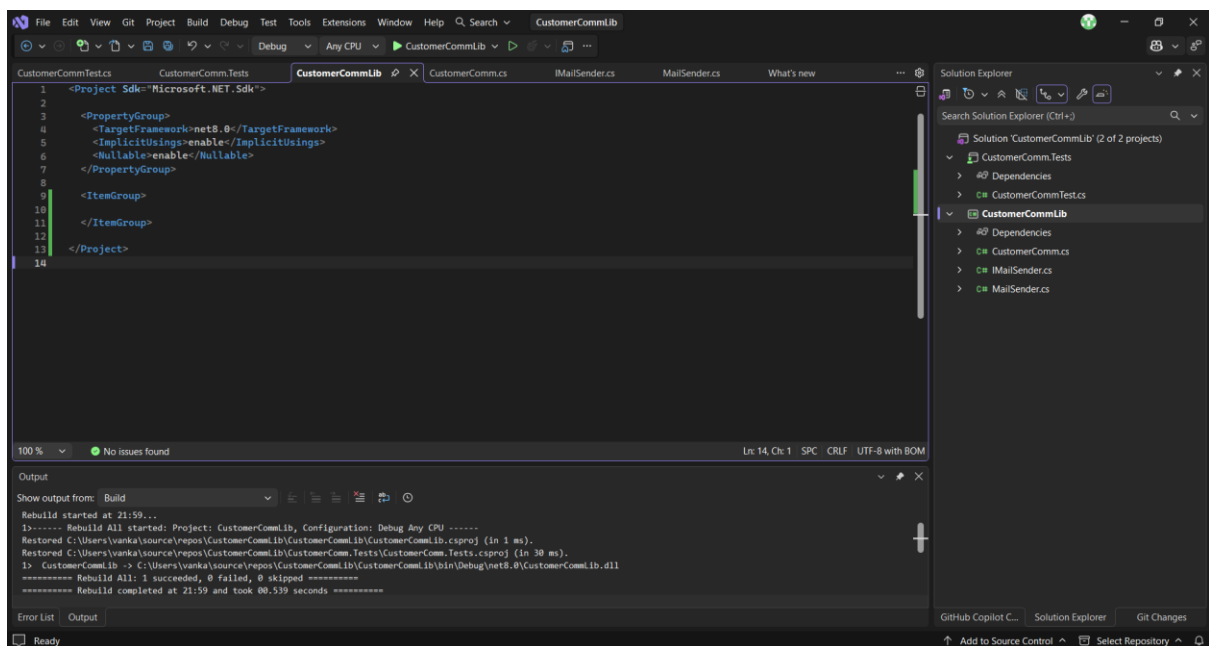
1. Open **Test Explorer**.
2. Click **Run All Tests**.
3. Verify that the test executes successfully.

Output

Passed: 1

Failed: 0

Skipped: 0



Result

Successfully created a testable mail communication module using dependency injection. Mock objects were created using Moq, and unit testing was performed using NUnit without sending actual emails. The test executed successfully and returned the expected result.

Conclusion

This exercise demonstrated how Moq can be used to mock external dependencies and how NUnit can be used to perform unit testing. The implementation ensured loose coupling, improved testability, and faster execution of test cases.